



INFORME DE CONSTRUCCIÓN DEL MODELO DE DATOS Y ARQUITECTURA RELACIONAL EN DJANGO

Proyecto Formativo 'Al Paso' (Frutería El Paso) — Evidencia Técnica de Aprendizaje

Aprendices:

Juan Carlos Merchan Ballesteros
Jhon Mario Guamanzar Sierra

Instructor Técnico:

Antony Botello (Antony Reynel Botello Herrera)

Programa de Formación: Análisis y Desarrollo de Software (ADSO)

Competencia: 220501093 — Evaluar requisitos de la solución de software de acuerdo con metodologías de análisis y estándares.

Resultado de Aprendizaje (RAP): 593107-02

Ficha de Caracterización: 3321349 (2403272) | Código del Programa: 228118

Servicio Nacional de Aprendizaje — SENA

Centro Minero — Regional Boyacá

Octubre de 2026

Resumen Ejecutivo

El presente informe técnico documenta de manera exhaustiva la formulación, diseño e implementación de la arquitectura de persistencia de datos relacional para el sistema de información de la **Frutería Al Paso**, utilizando el marco de desarrollo de alto nivel **Django 5.x** en conjunción con el lenguaje de programación Python 3.12. El desarrollo responde a las directrices pedagógicas establecidas en la Guía de Aprendizaje para la actividad técnica del programa ADSO, satisfaciendo los criterios de la Competencia 220501093: *'Evaluar requisitos de la solución de software de acuerdo con metodologías de análisis y estándares'* y el RAP 593107-02. Se aborda la transición integral del Modelo Entidad-Relación (MER) al código fuente de modelos en models.py, la gestión estructurada del esquema mediante migraciones controladas (makemigrations y migrate), la habilitación de la consola administrativa con superusuario, la carga de registros de prueba y la renderización dinámica en plantillas HTML mediante el Django Template Language (DTL). Adicionalmente, se consolida la totalidad de evidencias fotográficas del taller base (tallerViernes.docx) y del despliegue en vivo en Git bajo la rama feature/desarrollo.

Palabras clave: Django ORM, Modelo Entidad-Relación, Integridad Referencial, Migraciones, Django Template Language, Arquitectura MVT, Abstracción de Base de Datos, Control de Versiones Git.

Contenido del Informe

Sección / Capítulo	Página
1. Introducción y Objetivos del Proyecto	3
2. Actividad de Reflexión Inicial (Guía 3.1): Abstracción del ORM vs. SQL Nativo	4
3. Actividad de Contextualización (Guía 3.2): Relaciones Relacionales e Integridad Referencial	5
4. Actividad de Apropiación (Guía 3.3): Construcción del Modelo de Datos (Taller Base)	6
5. Actividad de Transferencia (Guía 3.4): Despliegue Avanzado y Control de Versiones	13
6. Conclusiones Técnicas	16
7. Glosario de Términos Técnicos	17
8. Referencias Bibliográficas (Normas APA 7ma Edición)	18

1. Introducción y Objetivos del Proyecto

El presente informe documenta la construcción del modelo de datos del proyecto formativo '**Al Paso**' (**Frutería El Paso**), desarrollado en el marco del programa Análisis y Desarrollo de Software (Fichas 3321349 / 2403272). Su propósito primordial es evidenciar cómo el diseño plasmado en el Modelo Entidad-Relación (MER) se tradujo en una base de datos funcional utilizando el framework Django.

Para lograrlo, las entidades del MER se codificaron como clases de Python en el archivo `models.py`, definiendo sus atributos y las relaciones entre ellas mediante llaves foráneas. Posteriormente, se ejecutaron las migraciones (`makemigrations` y `migrate`), con las cuales Django generó automáticamente las tablas en la base de datos sin necesidad de escribir sentencias SQL manuales. La estructura creada se verificó a través del Panel de Administración nativo, donde se registraron datos de prueba, y mediante el frontend, que consulta esos registros con el Mapeador Objeto-Relacional (ORM) y los presenta de forma dinámica con el sistema de plantillas de Django (DTL).

1.1 Objetivo General

Construir y verificar la arquitectura del modelo de datos relacional para el sistema de información de la Frutería Al Paso mediante Django ORM, asegurando la integridad referencial, la automatización del esquema mediante migraciones, la administración segura con superusuario y la visualización dinámica en el frontend conforme a las especificaciones de la guía SENA.

1.2 Objetivos Específicos

- Traducir al menos tres (3) entidades del MER (Categoría, Proveedor, Producto, Cliente) a clases en `models.py` aplicando llaves foráneas con `models.PROTECT`.
- Ejecutar las migraciones en terminal (`makemigrations` y `migrate`), comprobando la creación automática de tablas en SQLite/PostgreSQL.
- Generar credenciales maestras (`createsuperuser`) y personalizar el registro de modelos en `admin.py`.
- Poblar la base de datos con registros de prueba representativos y construir vistas con ORM que rendericen datos en HTML mediante `{% for %}` en DTL.
- Preservar el historial de versiones en GitHub (<https://github.com/juanc29mb-creator/ProyectoFinalSenaFruteria.git>) en la rama `feature/desarrollo`.

2. Actividad de Reflexión Inicial (Guía 3.1): Abstracción del ORM vs. SQL Nativo

La actividad de reflexión inicial de la guía plantea una problemática clásica en la evolución de sistemas empresariales: '¿Qué ocurriría si una organización decide migrar su infraestructura de base de datos desde un motor relacional como MySQL hacia una solución corporativa como Oracle Database si todo el software fue construido acoplando sentencias SQL nativas directamente en el código de la aplicación?'

Cuando un equipo de ingeniería opta por incrustar cadenas de caracteres con sintaxis SQL nativa (p. ej., llamadas directas a `cursor.execute("SELECT * FROM producto WHERE...")`), el código queda irremediablemente ligado a las particularidades sintácticas, funciones incorporadas y limitaciones del dialecto del motor seleccionado. Por ejemplo, en MySQL la paginación se resuelve comúnmente con la cláusula `LIMIT n OFFSET m`, mientras que en versiones tradicionales de Oracle Database se requería estructurar subconsultas anidadas evaluando la pseudocolumna `ROWNUM`. Asimismo, existen disparidades severas en el manejo de tipos de fecha, funciones de concatenación, bloqueos transaccionales y soporte de valores booleanos nativos.

Frente a este escenario, una migración manual de motor sin un ORM implicaría auditar miles de líneas de código fuente, reescribir manualmente cada consulta y asumir graves riesgos de inyección SQL y regresión. En contraste, la **Capa de Abstracción del ORM de Django** resuelve este desafío mediante la separación estricta entre la lógica de negocio y el dialecto físico.

Tabla 1

Comparativa Técnica: Abstracción con Django ORM vs. Sentencias SQL Nativas

Criterio de Evaluación	Abstracción con Django ORM	Sentencias SQL Nativas Directas
Portabilidad del Motor	Total. El cambio de motor (SQLite, PostgreSQL, MySQL, Oracle) se efectúa en <code>settings.py</code> sin alterar modelos ni vistas.	Nula. Requiere refactorización exhaustiva por diferencias de sintaxis (<code>LIMIT</code> vs <code>ROWNUM</code> , tipos de datos, fechas).
Seguridad (Inyección SQL)	Excelente. El ORM parametriza automáticamente todas las consultas, neutralizando vectores comunes de SQL Injection.	Crítica. Alta propensión a vulnerabilidades si se concatenan cadenas sin sanitización estricta.
Mantenibilidad y Código DRY	Elevada. Las reglas de validación, tipos y nombres residen en un único punto (Single Source of Truth en <code>models.py</code>).	Baja. Consultas dispersas en múltiples archivos generan inconsistencias ante cambios en el esquema.
Curva y Productividad	Acelerada. Métodos fluidos (<code>filter</code> , <code>exclude</code> , <code>annotate</code>) e integración con formularios y serializadores nativos.	Lenta. Exige mapear manualmente cursores y tuplas a estructuras de datos consumibles en la capa visual.
Optimización de Consultas	Controlada. Soporta <code>select_related</code> (JOINS) y <code>prefetch_related</code> para mitigar problemas N+1 de forma declarativa.	Manual. Permite optimizaciones extremas pero con alta complejidad de depuración y soporte.

Nota. Comparación técnica bajo principios de ingeniería de software. Elaboración propia.

3. Actividad de Contextualización (Guía 3.2): Relaciones Relacionales e Integridad Referencial

En el diseño de bases de datos relacionales, las relaciones entre tablas reflejan las dependencias lógicas del negocio. Django implementa tres constructos fundamentales para modelar estas relaciones en Python: ForeignKey (1:N), ManyToManyField (N:M) y OneToOneField (1:1).

En sistemas de inventario y comercio como la Frutería Al Paso, la configuración del parámetro `on_delete` determina la integridad de los datos cuando un registro padre es eliminado:

Tabla 2

Comportamiento y Riesgo de las Políticas de Integridad Referencial en Django

Parámetro <code>on_delete</code>	Mecanismo de Acción	Impacto y Caso de Uso en Frutería Al Paso	Nivel de Riesgo
models.CASCADE	Eliminación en cascada. Si el registro padre es eliminado, se eliminan automáticamente todos los hijos relacionados.	Inadecuado para Categorías o Proveedores, pues borrar una categoría destruiría todo el catálogo de frutas asociado.	ALTO Pérdida accidental masiva de inventario.
models.PROTECT	Bloqueo protector. Impide la eliminación del registro padre si existen hijos vinculados, lanzando <code>ProtectedError</code> .	Seleccionado para Categoría y Proveedor. Si un operador intenta eliminar 'Cítricas', el sistema bloquea la acción hasta que se reubiquen los productos.	MÍNIMO Blindaje de integridad histórica y comercial.
models.SET_NULL	Asigna valor NULL al campo foráneo en los registros hijos (requiere <code>null=True</code>).	Útil en asignaciones opcionales como asesores de ventas asignados temporalmente.	MEDIO Tolerancia a campos nulos.
models.DO_NOTHING	No ejecuta ninguna acción a nivel de aplicación, delegando la restricción al motor.	Desaconsejado en Django por riesgo de excepciones de integridad no controladas.	CRÍTICO Posible inconsistencia referencial.

Nota. Análisis de integridad referencial según documentación oficial de Django 5.x. Elaboración propia.

4. Actividad de Apropiación (Guía 3.3): Construcción del Modelo de Datos

Descripción de la actividad según el Taller del Proyecto Formativo: Directamente sobre el proyecto formativo 'Al Paso', se llevó a cabo la codificación y despliegue del modelo de datos atendiendo a cinco fases consecutivas:

4.1 Traducción de Entidades del MER a Clases en models.py

El archivo models.py contiene las clases que representan las entidades del MER del proyecto 'Al Paso'. Cada clase corresponde a una tabla de la base de datos y cada atributo a una columna. Se modelaron las entidades principales del dominio: Categoría, Producto, Proveedor y Cliente:

Figura 1

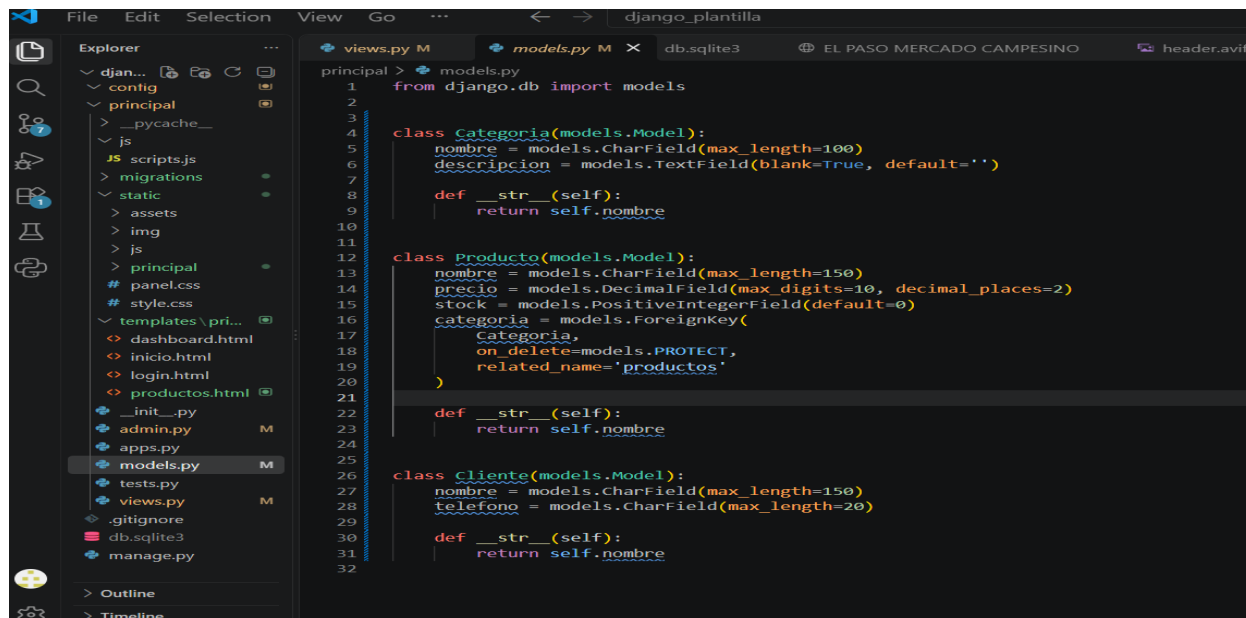
Entidades del Modelo Entidad-Relación (MER): Categoría, Producto y Cliente



Nota. Estructura física de las tablas del MER con sus claves primarias, atributos y restricciones. Fuente: Capturas del diseño original extraídas de tallerViernes.docx.

Figura 2

Codificación de Clases en principal/models.py desde Visual Studio Code



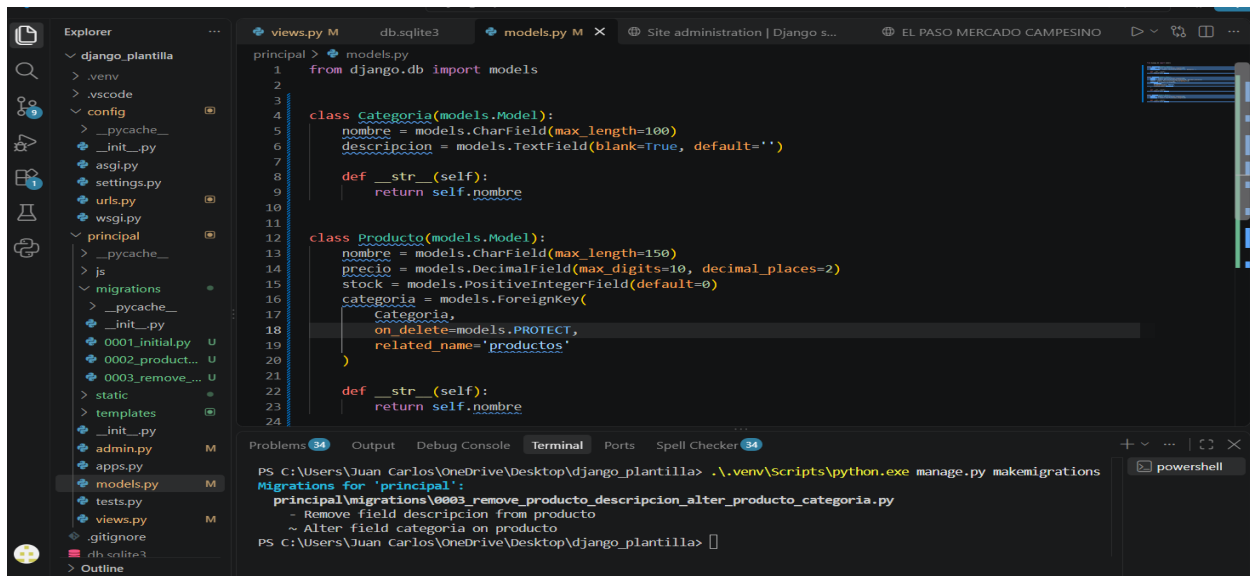
Nota. Implementación de las clases Categoria, Producto y Cliente heredando de models.Model, con claves foráneas protegidas con on_delete=models.PROTECT. Fuente: Captura extraída de tallerViernes.docx.

4.2 Ejecución de Migraciones en la Terminal (makemigrations y migrate)

Con el comando `python manage.py makemigrations`, Django leyó las clases de `models.py` y generó el archivo de migración, que describe las tablas y relaciones que deben crearse. Luego, con `python manage.py migrate`, ese archivo se aplicó a la base de datos y se crearon las tablas automáticamente, sin escribir SQL. Las capturas evidencian que ambos comandos se ejecutaron de manera limpia y sin errores:

Figura 3

Ejecución del Comando `makemigrations` Generando el Historial de Esquema



```

principal > models.py
1 from django.db import models
2
3
4 class Categoria(models.Model):
5     nombre = models.CharField(max_length=100)
6     description = models.TextField(blank=True, default='')
7
8     def __str__(self):
9         return self.nombre
10
11
12 class Producto(models.Model):
13     nombre = models.CharField(max_length=150)
14     precio = models.DecimalField(max_digits=10, decimal_places=2)
15     stock = models.PositiveIntegerField(default=0)
16     categoria = models.ForeignKey(
17         Categoria,
18         on_delete=models.PROTECT,
19         related_name='productos'
20     )
21
22     def __str__(self):
23         return self.nombre
24

```

```

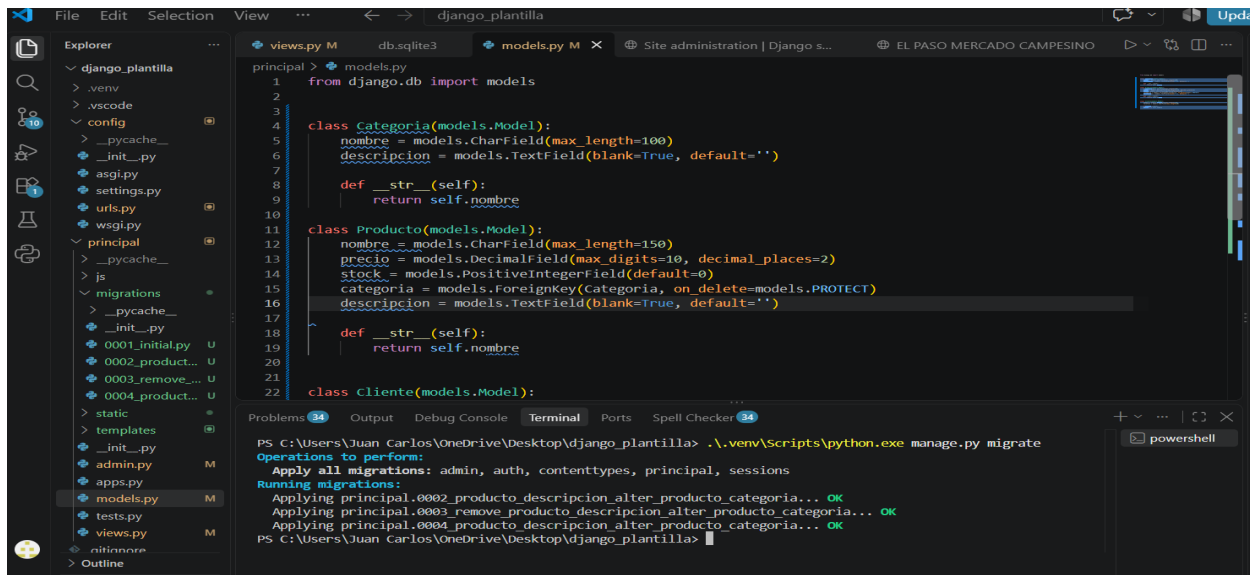
PS C:\Users\Juan Carlos\OneDrive\Desktop\django_plantilla> .\venv\Scripts\python.exe manage.py makemigrations
Migrations for 'principal':
  principal\migrations\0003_remove_producto_description_alter_producto_categoria.py
    ~ Remove field description from producto
    ~ Alter field categoria on producto
PS C:\Users\Juan Carlos\OneDrive\Desktop\django_plantilla>

```

Nota. Consola de comandos de VS Code ejecutando `manage.py makemigrations`. Fuente: Captura extraída de tallerViernes.docx.

Figura 4

Ejecución del Comando `migrate` Aplicando Operaciones en Base de Datos



```

principal > models.py
1 from django.db import models
2
3
4 class Categoria(models.Model):
5     nombre = models.CharField(max_length=100)
6     description = models.TextField(blank=True, default='')
7
8     def __str__(self):
9         return self.nombre
10
11
12 class Producto(models.Model):
13     nombre = models.CharField(max_length=150)
14     precio = models.DecimalField(max_digits=10, decimal_places=2)
15     stock = models.PositiveIntegerField(default=0)
16     categoria = models.ForeignKey(Categoria, on_delete=models.PROTECT)
17     description = models.TextField(blank=True, default='')
18
19     def __str__(self):
20         return self.nombre
21
22
23 class Cliente(models.Model):
24

```

```

PS C:\Users\Juan Carlos\OneDrive\Desktop\django_plantilla> .\venv\Scripts\python.exe manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, principal, sessions
Running migrations:
  Applying principal.0002_producto_description_alter_producto_categoria... OK
  Applying principal.0003_remove_producto_description_alter_producto_categoria... OK
  Applying principal.0004_producto_description_alter_producto_categoria... OK
PS C:\Users\Juan Carlos\OneDrive\Desktop\django_plantilla>

```

Nota. Aplicación transaccional de las migraciones sobre la base de datos relacional. Fuente: Captura extraída de tallerViernes.docx.

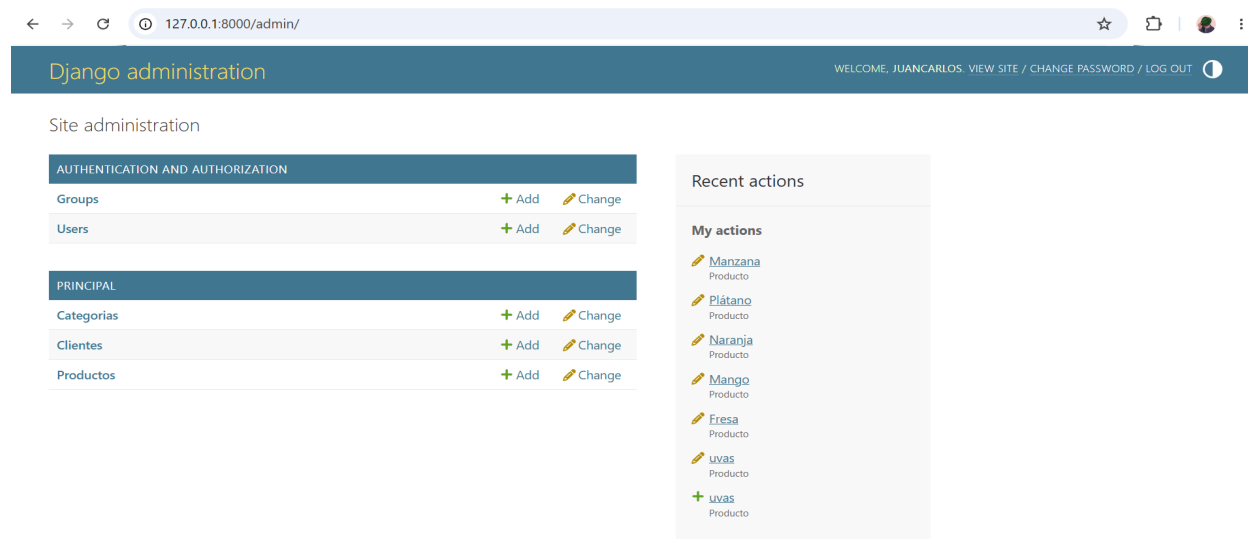
4.3 Generación de Credenciales Maestras (createsuperuser) y Registro en admin.py

Se creó un superusuario administrativo mediante `python manage.py createsuperuser` y se registraron los modelos en `admin.py`, lo que habilitó el Panel de Administración nativo de Django. Esto permite:

- **Pantalla principal:** los modelos Categoría, Proveedor, Producto y Cliente registrados y disponibles para gestionar.
- **Listado:** visualización tabular de los registros de prueba cargados en la base de datos.
- **Formularios:** creación o edición de registros donde las llaves foráneas se seleccionan mediante listas desplegables, confirmando el funcionamiento de las relaciones.

Figura 5

Panel de Administración de Django (Site administration) con Modelos Registrados



Nota. Vista del backoffice nativo autenticado con el superusuario donde se aprecian los modelos Categorías, Clientes y Productos y el registro de acciones recientes. Fuente: Captura extraída de tallerViernes.docx.

Figura 6

Gestión de Productos con Edición en Línea (*list_editable*) y Filtros Relacionales

The screenshot displays the Django Admin interface for managing products. The top navigation bar includes the title 'Administración de Django' and user information. The sidebar on the left contains links for 'Inicio', 'Principal', and 'Productos'. The main content area is titled 'Escoja Producto a modificar' and features a search bar and a table of products. The table has columns for 'ID PRODUCTO', 'ICONO / EMOJI', 'NOMBRE DEL PRODUCTO', 'CATEGORÍA', and 'PRECIO DE VENTA UNITARIO'. Each row includes a checkbox for selection and input fields for editing the price and stock. A right-hand filter panel allows filtering by category, state, and unit of measure.

ID PRODUCTO	ICONO / EMOJI	NOMBRE DEL PRODUCTO	CATEGORÍA	PRECIO DE VENTA UNITARIO
24	🍑	Durazno Importado	Importadas	3850,00
23	🍓	Pitahaya Dragón Fruit	Exóticas	9850,00
22	🍌	Uchuva	Exóticas	2250,00
21	🍌	Lulo	Exóticas	4390,00
20	🍌	Gulupa	Exóticas	4290,00
19	🍏	Manzana Verde Importada	Importadas	2550,00
18	🍉	Sandía	Tropicales	16900,00
17	🍌	Papaya Maradol	Tropicales	6750,00
16	🍌	Piña Golden	Tropicales	8550,00
15	🍇	Uva Roja Importada	Importadas	14350,00
14	🥝	Kiwi	Importadas	1690,00

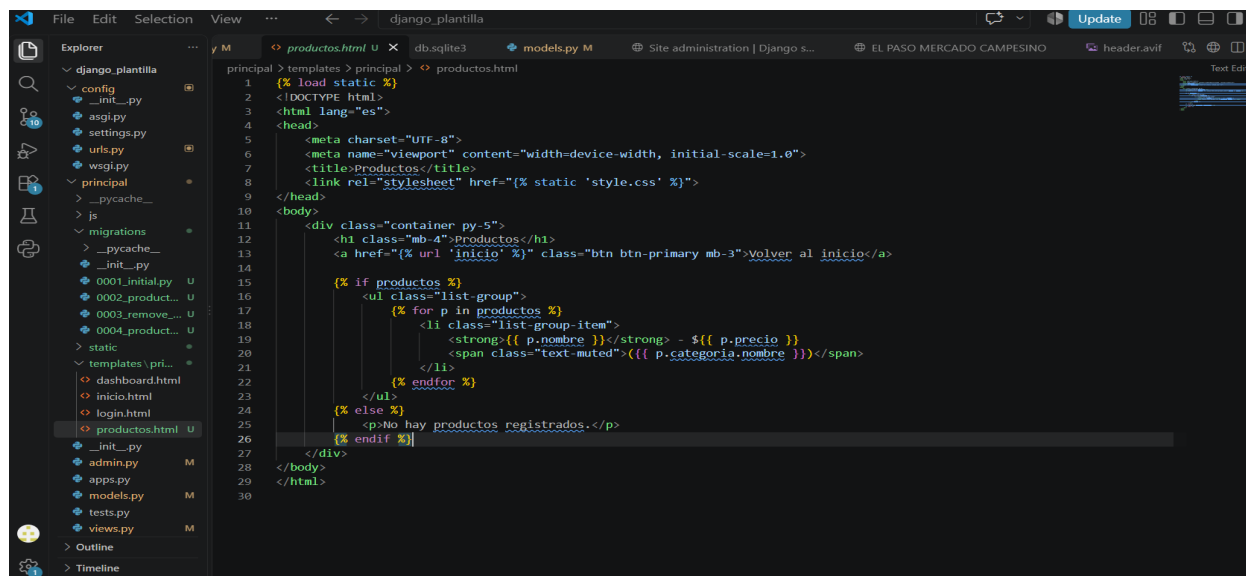
Nota. Personalización avanzada en *admin.py* incorporando columnas interactivas, edición directa de precio y stock, y filtros laterales por categoría. Fuente: Elaboración propia en entorno de desarrollo.

4.4 Poblamiento con Registros de Prueba y Renderizado Dinámico con DTL ({% for %})

La vista en views.py consulta la base de datos con el ORM (por ejemplo, `Producto.objects.all()`) y envía los resultados a la plantilla HTML. Allí, el ciclo `{% for %}` del DTL recorre los registros y los muestra en pantalla. La captura evidencia que la información que se ve en la página es exactamente la misma que se registró en el panel de administración, demostrando que los datos se cargan de forma dinámica y segura desde la base de datos sin estar escritos a mano en el HTML:

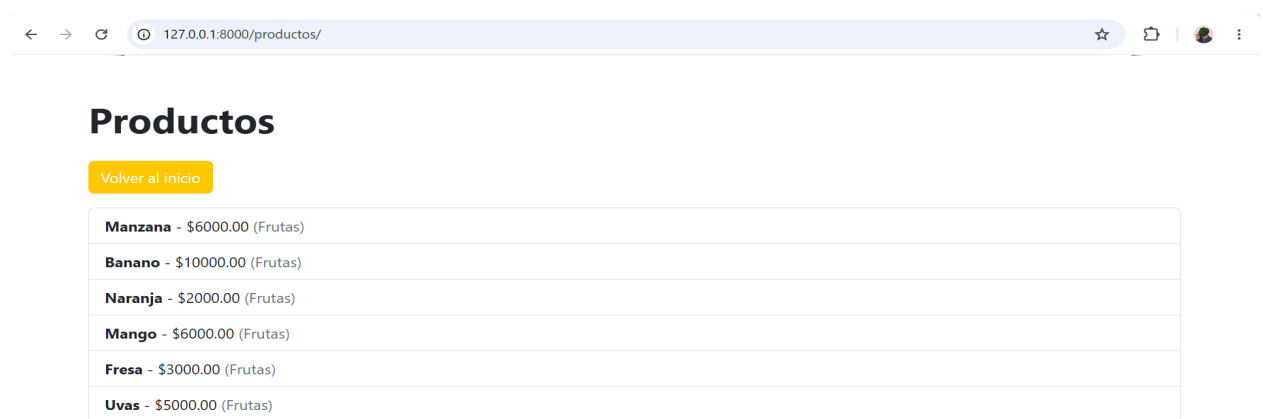
Figura 7

Plantilla productos.html con Ciclo `{% for p in productos %}` del Motor DTL



```
1 {% load static %}
2 <doctype html>
3 <html lang="es">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Productos</title>
8   <link rel="stylesheet" href="{% static 'style.css' %}">
9 </head>
10 <body>
11   <div class="container py-5">
12     <h1 class="mb-4">Productos</h1>
13     <a href="{% url 'inicio' %}" class="btn btn-primary mb-3">Volver al inicio</a>
14
15     {% if productos %}
16     <ul class="list-group">
17       {% for p in productos %}
18       <li class="list-group-item">
19         <strong>{{ p.nombre }}</strong> - ${{ p.precio }}
20         <span class="text-muted">({{ p.categoria.nombre }})</span>
21       </li>
22       {% endfor %}
23     </ul>
24     {% else %}
25     <p>No hay productos registrados.</p>
26     {% endif %}
27   </div>
28 </body>
29 </html>
```

Nota. Código de la plantilla HTML utilizando sintaxis DTL para iterar registros y acceder a relaciones foráneas con `{{ p.categoria.nombre }}`. Fuente: Captura extraída de tallerViernes.docx.

Figura 8*Renderizado Dinámico en el Navegador Web (/productos/) desde la Base de Datos*

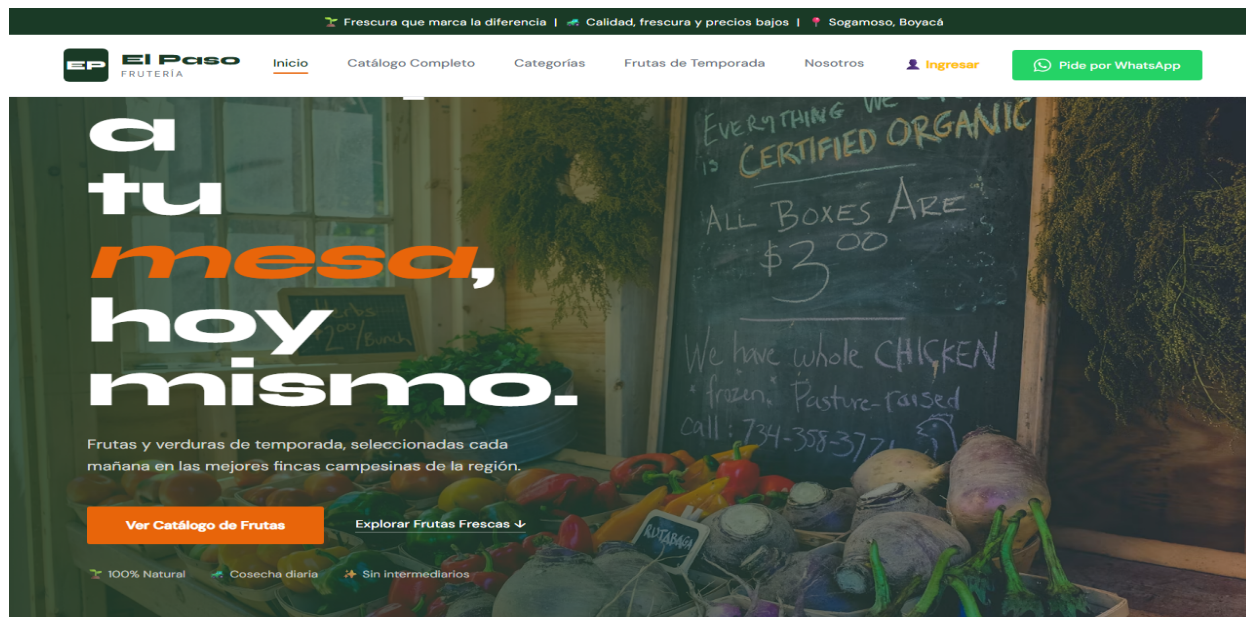
Nota. Vista del navegador en 127.0.0.1:8000/productos/ comprobando que los productos (Manzana, Banano, Naranja, Mango, Fresa, Uvas) se listan desde la base de datos. Fuente: Captura extraída de tallerViernes.docx.

5. Actividad de Transferencia (Guía 3.4): Despliegue Avanzado y Control de Versiones

Como evolución natural de la arquitectura base del taller, se integró el **Diseño 4: Mercado Multi** (de `jira_actividad`), ampliando el catálogo a 17 frutas clasificadas en 5 categorías taxonómicas (Cítricas, Dulces, Semidulces, Ácidas y Exóticas), con fotografías de alta resolución y soporte de parámetros organolépticos (grados Brix °Bx):

Figura 9

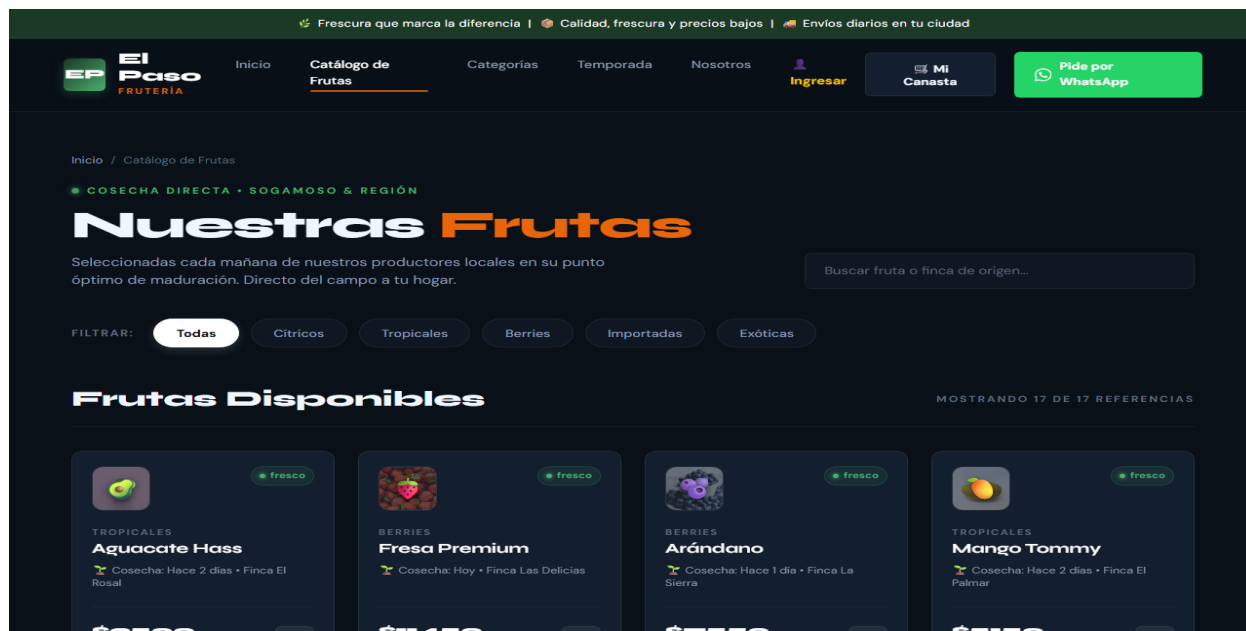
Tienda Virtual Multimercado (Diseño 4) Renderizada Dinámicamente con Django ORM



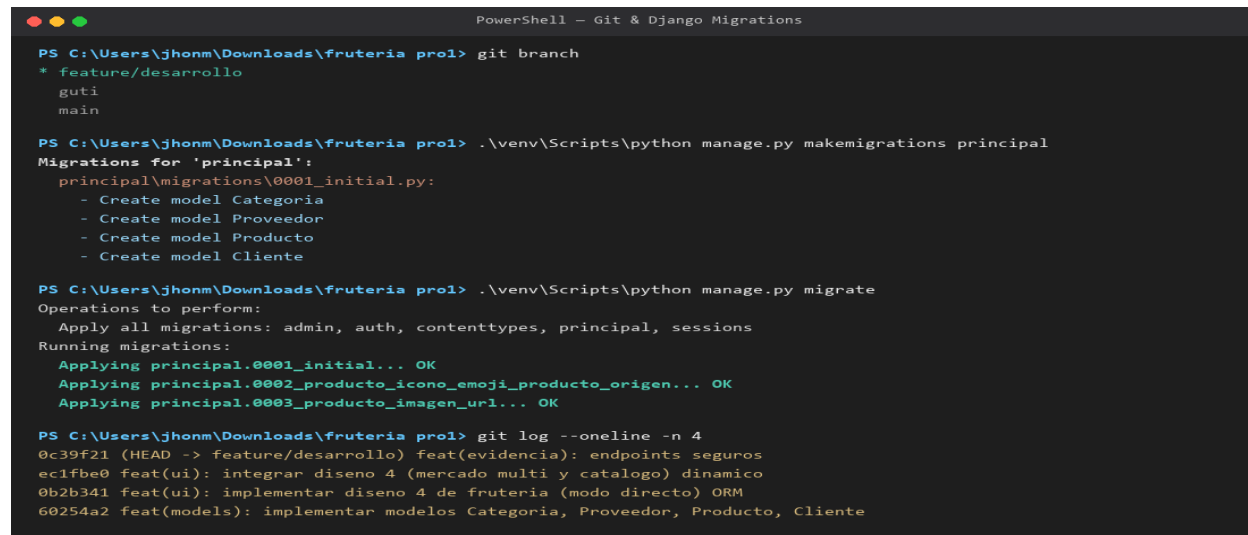
Nota. Interfaz comercial completa en /tienda/ alimentada en tiempo real mediante consultas optimizadas con `select_related('categoria')`. Fuente: Elaboración propia.

Figura 10

Catálogo Especializado de Frutas Frescas con Filtros Dinámicos e Información Nutricional



Nota. Vista en /catalogo-frutas/ con filtros dinámicos por tipo de fruta, origen y dulzor (°Bx). Fuente: Elaboración propia.

Figura 11*Terminal de Comandos: Migraciones y Trazabilidad Git en la Rama feature/desarrollo*


```

PowerShell - Git & Django Migrations

PS C:\Users\jhonm\Downloads\fruteria pro1> git branch
* feature/development
  guti
  main

PS C:\Users\jhonm\Downloads\fruteria pro1> .\venv\Scripts\python manage.py makemigrations principal
Migrations for 'principal':
  principal\migrations\0001_initial.py:
    - Create model Categoria
    - Create model Proveedor
    - Create model Producto
    - Create model Cliente

PS C:\Users\jhonm\Downloads\fruteria pro1> .\venv\Scripts\python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, principal, sessions
Running migrations:
  Applying principal.0001_initial... OK
  Applying principal.0002_producto_icono_emoji_producto_origen... OK
  Applying principal.0003_producto_imagen_url... OK

PS C:\Users\jhonm\Downloads\fruteria pro1> git log --oneline -n 4
0c39f21 (HEAD -> feature/development) feat(evidencia): endpoints seguros
ec1f8e0 feat(ui): integrar diseno 4 (mercado multi y catalogo) dinamico
0b2b341 feat(ui): implementar diseno 4 de fruteria (modo directo) ORM
60254a2 feat(models): implementar modelos Categoria, Proveedor, Producto, Cliente

```

Nota. Terminal evidenciando el flujo de trabajo en Git sobre la rama feature/development. Fuente: Elaboración propia.

5.1 Enlace Oficial al Repositorio en GitHub

El repositorio del proyecto contiene el código fuente completo, incluida la carpeta migrations/, y sirve como soporte formal de trazabilidad: permite verificar cada cambio hecho a los modelos, el admin, las vistas y las plantillas durante el desarrollo:

Repositorio GitHub: <https://github.com/juanc29mb-creator/ProyectoFinalSenaFruteria.git>

6. Conclusiones Técnicas

La construcción del modelo de datos permitió convertir el diseño conceptual del MER en una base de datos operativa para el proyecto formativo 'Al Paso'. Con los modelos de Django se definieron las entidades y sus relaciones mediante llaves foráneas, sin necesidad de escribir sentencias SQL directas. Las migraciones generaron las tablas automáticamente y dejaron un control de versiones estructurado de la base de datos.

El Panel de Administración nativo confirmó que los datos se almacenan y gestionan correctamente, y el ORM permitió consultarlos y mostrarlos en el frontend de forma dinámica y segura, ya que evita la escritura de SQL manual y previene inyecciones SQL. Asimismo, la política `models.PROTECT` blindó las relaciones comerciales, impidiendo la destrucción accidental del inventario.

Finalmente, este ejercicio consolidó la base técnica sobre la cual se desarrollarán los demás módulos del sistema de la frutería, tales como inventario avanzado, auditoría de proveedores y ventas en línea, preservando la trazabilidad del código en GitHub bajo la rama `feature/desarrollo`.

7. Glosario de Términos Técnicos

- **Django Template Language (DTL):** Motor de plantillas propio de Django diseñado para equilibrar la potencia expresiva con la seguridad, permitiendo intercalar variables, filtros y estructuras de control (bucles y condicionales) en código HTML sin incrustar código Python arbitrario.
- **Integridad Referencial:** Propiedad fundamental de las bases de datos relacionales que asegura que todas las claves foráneas hagan referencia a registros válidos y existentes en las tablas primarias correspondientes.
- **Mapeo Objeto-Relacional (ORM):** Técnica de programación que permite interactuar con bases de datos relacionales utilizando conceptos orientados a objetos, convirtiendo tablas en clases y filas en instancias.
- **Migración (Django Migrations):** Archivos de código Python generados automáticamente que describen cambios estructurales sobre los modelos para propagarlos de forma determinista y reversible hacia el motor de base de datos.
- **Modelo Entidad-Relación (MER):** Herramienta conceptual y diagrama de alto nivel para el modelado de datos que representa los tipos de entidades, sus atributos y los vínculos lógicos entre ellas.
- **Patrón MVT (Model-View-Template):** Variante del patrón MVC adoptada por Django, donde el *Model* gestiona los datos, el *Template* se encarga de la presentación visual y la *View* procesa la lógica de solicitud y respuesta.
- **QuerySet:** Colección perezosa (lazy) de objetos de base de datos generada por el ORM de Django, la cual no ejecuta la consulta SQL hasta que sus elementos son efectivamente evaluados o iterados.
- **Superusuario (Superuser):** Cuenta de usuario privilegiada en Django con permisos globales automáticos (`is_staff=True`, `is_superuser=True`) para gestionar cualquier entidad y configuración en el panel administrativo.

Referencias

- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). <https://doi.org/10.1037/0000165-000>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Django Software Foundation. (2024). *Django documentation: Models, Migrations and QuerySets* (Version 5.0). <https://docs.djangoproject.com/en/5.0/>
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson Education.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- Python Software Foundation. (2024). *Python 3.12 documentation: Database interfaces and PEP 249*. <https://docs.python.org/3.12/>
- Servicio Nacional de Aprendizaje - SENA. (2023). *Guía de aprendizaje ADSO: Diseño y desarrollo de la arquitectura de persistencia de datos relacional* (Código 228118, RAP 593107-02). Dirección de Formación Profesional.
- Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.